Intensity Estimation

Raj Kundi

2026-08-25

Table of contents

1 Market maker problem

A market maker looking to post limit orders into the book, using a standard market making models (i.e. Guéant et al. (2013)), needs to create a model for the fill rate of her limit orders.

1.1 Framework

Suppose she places limit orders continuously at a spread δ_t^a and δ_t^b on the bid and ask side respectively in a venue with mid price S_t. Then using a two point process denoted N_t^a and N_t^b for the ask and bid orders respectively, we can model the number of market orders that have lifted her limit orders. The intensities of the processes N_t^a and N_t^b are denoted λ_t^a and λ_t^b respectively.

The main assumption of the Avellaneda-Stoikov (AS) model (Avellaneda and Stoikov 2008) and many models in (Guéant et al. 2013) is:

$$\lambda_t^b = \Lambda^b(\delta_t^b)\chi_{q_{t-}<Q} \quad \text{and} \quad \lambda_t^a = \Lambda^a(\delta_t^a)\chi_{q_{t-}>-Q},$$

where:

- $\delta_t^a = S_t^a - S_t$ and $\delta_t^b = S_t - S_t^b$, where S_t^a and S_t^b are the posted limit order prices for the ask and bid respectively;
- both Λ^a and Λ^b are two positive and nonincreasing functions, and
- $Q \in \mathbb{N}$ represents the maximum inventory (long or short). (In practice, this means that she stops placing any more trades if the inventory skews far away from a target.)

A further assumption in the AS model is:

$$\Lambda^b(\delta) = \Lambda^a(\delta) = Ae^{-\kappa\delta},$$

where $A > 0$ models the assets liquidity and $\kappa > 0$ measures the price sensitivity of market participants.

1.2 Interpretation of fill rate model

The intensity $\Lambda(\delta)$ is the rate at which a limit order resting at distance δ from the mid get filled, which is the rate of market orders whose price execution reaches *at least* δ. It is not the density of fills at the level δ. Fundamentally, $\Lambda(\delta)$ is the rate at which any limit order resting at or beyond a distance δ from the reference price gets filled.

The parameter A is theoretically equal to $\Lambda(0)$ which practically is the fill rate at the reference price (which is nonsensical since this is a depth at which no limit order can actually rest).

Both points are corrected in Section 3.4.

2 Estimating intensity functions

We do not clean the data for large trades, wide spreads, extreme volumes or volatile episodes. Removing those would be counterintuitive since it is a regime we are trying to model. Instead of discarding them, we split the data into regimes based on the volatility and estimate the parameters (A, κ) within each regime in Section 4.

The only rows dropped are ones where the data is malformed (i.e. the pre-mid is on the wrong side of the execution price depending on the if the buyer is a maker or taker).

```
# Data source in ./data folder
SYMBOL <- "SOLUSD_PERP"
MONTH <- "2024-08"
TICK_SIZE <- 0.001  # USD. The model is estimated in USD; the tick grid only
                    # sets the bin width and gives a secondary readout of kappa.

# Fill -> order aggregation. Binance stamps every fill of one aggressing order
# with the same millisecond, but a sweep may occasionally straddle a boundary.
ORDER_GAP_MS <- 5

# Distance-from-mid binning for the intensity regressions. Bins are laid out on
# the tick grid (prices are discrete, so anything else aliases adjacent levels
# together unevenly) but their width is carried in USD, which is the unit delta
# is measured in throughout.
BIN_TICKS <- 0.5  # bin width in ticks
BIN_SIZE <- BIN_TICKS * TICK_SIZE  # bin width in USD
TRIM_Q <- 0.99  # fit is restricted to distance_from_mid <= this pooled quantile

# Volatility regime construction.
BAR_MINUTES <- 5  # bar length for the realised-variance grid
VOL_WINDOW <- 15  # trailing bars in the realised-volatility window
N_VOL_BINS <- 4  # number of volatility regimes (quantile cut)
```

```
trades <- fread(sprintf("data/%s-trades-%s.csv", SYMBOL, MONTH))
bookTicker <- fread(
  sprintf("data/%s-bookTicker-%s.csv", SYMBOL, MONTH),
  select = c("best_bid_price", "best_ask_price", "transaction_time")
```

```
)

trades[, ts := as.POSIXct(time / 1000, origin = "1970-01-01", tz = "UTC")]
trades[, date := as.Date(ts)]

# Half the quoted spread: what a marketable order concedes just by crossing.
bookTicker[, half_spread := (best_ask_price - best_bid_price) / 2]
bookTicker[, ts := as.POSIXct(
  transaction_time / 1000,
  origin = "1970-01-01",
  tz = "UTC"
  )]

setorder(trades, time, id)
setorder(bookTicker, transaction_time)
```

The sample contains 7,923,220 fills and 74,641,633 orderbook updates spanning 2024-08-01 to 2024-08-31.

2.1 Attaching the reference price pre-trade

For each trade in the trades data, we add the most recent mid price strictly before the trade was executed using the order book tick data.

```
setkey(bookTicker, transaction_time)

prev_quote <- bookTicker[
  trades,
  on = .(transaction_time < time),
  mult = "last",
  .(best_bid_price, best_ask_price)
]

trades[, `:=`(pre_bid = prev_quote$best_bid_price,
              pre_ask = prev_quote$best_ask_price)]

rm(prev_quote) # no longer needed.

trades[, pre_mid := (pre_bid + pre_ask) / 2]
```

2.2 Fills must be aggregated into orders

A single market order that walks the limit order book is reported by binance as several trades sharing a timestamp. The intensity model $\Lambda(\delta)$ is the arrival rate of market orders that reach a limit order with spread δ. A sweep that consumes five levels is one such arrival, characterised by the deepest level it touched. By not aggregating the trades, we would double count the shallower levels and overestimate the likelihood of lifting limit orders with a smaller spread.

3 Analysis

3.1 Distance from the mid/reference price, aggressor side and the aggregation to orders

The attribute/field `distance_from_mid` is the signed distance from the pre-trade mid-price to the execution price of a trade in the trades data, measured in USD.

The unit is not a presentational choice. In the AS and Guéant solutions δ is a *price* offset — $\delta_t^a = S_t^a - S_t$ is a number of dollars — so κ carries units of USD^{-1} and the optimal-quote formulas consume it in exactly that form. Estimating in ticks and dividing by `TICK_SIZE` at the end gives the same answer, but it puts a unit conversion at the one point where a mistake is least visible: κ is a slope, so a wrong conversion still produces a plausible-looking fit. So δ is carried in USD from the raw trade onwards, and κ is estimated per USD directly.

A per-tick figure $\kappa_{\text{tick}} = \kappa \times$ `TICK_SIZE` is reported alongside as a reading aid, since the exponential decays over a handful of ticks and that is the easier scale to eyeball. The tick grid also survives in the binning: bins are laid out as multiples of a tick and only then expressed in USD.

A is unaffected by any of this. It is a rate in orders per second and carries no price units, so it is the same number whichever unit δ is measured in.

```
# is_buyer_maker == TRUE means the maker bought and therefore
# the taker SOLD into the bid.
# That is the process N^b: it is the market maker's bid that gets lifted.
trades[, side := factor(
  fifelse(is_buyer_maker, "bid (market sells)", "ask (market buys)")
)]

trades[
  ,
  distance_from_mid := fifelse(
    is_buyer_maker,
```

```
    pre_mid - price,
    price - pre_mid)
]

BAR_MS <- BAR_MINUTES * 60 * 1000  # converts time to ms
trades[, bar := floor(time / BAR_MS) * BAR_MS]


trades <- trades[distance_from_mid > 0]  # clean up
```

For each market order (which may be multiple trades with the same timestamp in `trades`), we aggregate the trades to get the deepest price hit, along with relevant metrics.

```
trades[, order_group := floor(time / ORDER_GAP_MS) * ORDER_GAP_MS]

orders <- trades[
  !is.na(distance_from_mid) & distance_from_mid > 0,
    .(distance_from_mid = max(distance_from_mid),
    n_fills = .N,
    bar = first(bar),
    date = first(date)),
  by = .(order_group, side)
]

T_obs <- as.numeric(max(trades$time) - min(trades$time)) / 1000

orders[
  , .(
    orders = .N,
    fills_per_order = mean(n_fills),
    rate_per_sec = .N / T_obs
  ),
  by = side
]
```

```
                side  orders fills_per_order rate_per_sec
              <fctr>   <int>           <num>        <num>
1:  ask (market buys) 1473077        2.608541    0.5499859
2: bid (market sells) 1501501        2.707754    0.5605983
```

The `fills_per_order` measures how much the shallow bins would have been overestimated in modelling.

3.2 Model estimation

The goal is to model $\Lambda(\delta) = Ae^{-\kappa\delta}$, which is just the orders reaching at least δ away from the mid price.

3.2.1 The tail identity, and why it is not what we regress

The expression $\Lambda(\delta) = Ae^{-\kappa\delta}$ is the intensity of the two point process modelling the base quantity of assets bought or sold. It decomposes into two pieces:

- λ_{tot}, the total arrival rate of all market orders (orders per second), and
- $\mathbb{P}(p_x \geq \delta)$, the probability that a randomly arriving market order x's deepest price p_x reaches at least distance δ from the mid.

The rate at which orders reach that depth is just the product of the two, which gives:

$$\Lambda(\delta) = \lambda_{\text{tot}} \cdot \mathbb{P}(p_x \geq \delta).$$

This identity is what Λ *means*, and it is the object plotted in Figure 2. It is also the obvious thing to regress on, and that is the trap: the counts entering it are nested. An order that reaches three ticks deep also reached two ticks, and every order that reached two ticks also reached one, so the observations at successive depths are near-copies of one another rather than independent draws. Regressing on them produces heavily autocorrelated residuals, and correlated residuals make the standard errors meaningless — the confidence intervals come out far too narrow while the point estimate looks perfectly reasonable.

The fix is to regress on something disjoint instead. Differencing the tail gives the density, whose bins partition the data rather than nesting it, and it carries exactly the same κ.

3.2.2 Fitting the density

Since the density of the `distance_from_mid` is the negative derivative of 1 minus the cumulative density function, we can see that

$$f(\delta) = -\Lambda'(\delta)$$

and if $\Lambda(\delta) = Ae^{-\kappa\delta}$, then:

$$f(\delta) = A\kappa e^{-\kappa\delta}$$

This is a *rate* density rather than a probability density: it is the arrival rate of orders whose furthest distance from the mid is δ, per unit of depth, in the infinitesimal sense. Integrating it over all depths returns λ_{tot} orders per second, not 1. That distinction is what keeps A in orders per second rather than leaving it dimensionless.

Instead of computing the tail counts directly, by binning the data into intervals with a fixed width w and centered at δ_k,

$$[\delta_k - w/2, \delta_k + w/2).$$

The expected count of orders in a bin k is then just

$$\mathbb{E}[N_k] = T_b \int_{\delta_k - w/2}^{\delta_k + w/2} f(\delta)\, d\delta$$

where T_b is the observation/bins window length. (NB: we are talking about the time length.)

For exponential density, where $f(\delta) = A\kappa e^{-\kappa\delta}$:

$$\begin{aligned}
\mathbb{E}[N_k] &= T_b \cdot A\kappa \textstyle\int_{\delta_k - w/2}^{\delta_k + w/2} e^{-\kappa\delta}\, d\delta, \\
&= T_b \cdot A\kappa \left[-\tfrac{1}{\kappa} e^{-\kappa\delta}\right]_{\delta_k - w/2}^{\delta_k + w/2}, \\
&= T_b \cdot A \left(e^{-\kappa(\delta_k - w/2)} - e^{-\kappa(\delta_k + w/2)}\right), \\
&= T_b \cdot A \cdot e^{-\kappa\delta_k} \left(e^{\kappa w/2} - e^{-\kappa w/2}\right), \\
&= T_b \cdot A \cdot e^{-\kappa\delta_k} \cdot 2\sinh\left(\tfrac{\kappa w}{2}\right).
\end{aligned}$$

Note that $2\sinh(x) = e^x - e^{-x}$ and the antiderivative of $e^{-\kappa\delta}$ is $-\frac{1}{\kappa}e^{-\kappa\delta}$.

Hence, taking logs and rearranging gives

$$\log\left(\frac{\mathbb{E}[N_k]}{T_b}\right) = \log\left(2\sinh\left(\frac{\kappa w}{2}\right)\right) + \log(A) - \kappa\delta_k$$

So, we could regress $\log(N_k/T_b)$ on δ_k using a Poisson GLM:

$$\text{log-rate} = \text{intercept} + \text{slope} \times \delta_k$$

- slope is $-\kappa$, which gives the price sensitivity, and
- the intercept $\log(2\sinh(\kappa w/2)) + \log(A)$ gives A up to a bin width correction.

3.2.3 Two corrections to the level

So the estimator is a single Poisson GLM on disjoint half-tick bins. The density and the tail decay at the same rate, so nothing about κ is given up by fitting the density instead of the tail — the slope is $-\kappa$ and can be read straight off.

The intercept is a different matter. It is *not* $\log A$, and the two reasons why are both easy to skip.

The bin-width factor. The regression predicts the count in a bin of finite width, not the value of the density at a point. Integrating over one bin,

$$\mathbb{E}[N_k]/T_b \;=\; A\, e^{-\kappa\delta_k}\left(e^{\kappa w/2} - e^{-\kappa w/2}\right) \;=\; \underbrace{2\sinh\left(\tfrac{\kappa w}{2}\right)}_{\text{bin-width factor}} A\, e^{-\kappa\delta_k},$$

so the fitted intercept is $\log(2\sinh(\kappa w/2)\,A)$, not $\log A$. The factor depends only on the dimensionless product κw, so it is the same number whichever unit δ is carried in: a half-tick bin with κ of order 1 per tick, and the same bin written as $w = 0.0005$ USD with κ of order 1000 per USD, both give $\kappa w \approx 0.5$ and a factor near 0.5, which causes a two-fold error in the level.

Left truncation. The shallowest observable distance from the mid is half the quoted spread, $\delta_{\min}$, so it can never be zero. This does not bias κ — the exponential is memoryless, and the fit simply starts further out — but it does mean the raw order rate $\lambda_{\text{tot}} = N_{\text{side}}/T_b$ is $\Lambda(\delta_{\min})$ rather than $\Lambda(0) = A$. Extrapolating back,

$$A \;=\; \lambda_{\text{tot}}\, e^{\kappa\delta_{\min}},$$

which uses only the total count and the truncation point, and so provides a check on the level that is independent of the fitted line's shape. Section 3.4 runs it as exactly that: a consistency test on the GLM's intercept, not a competing estimate.

Both corrections raise the level. The intercept has to be divided by $2\sinh(\kappa w/2)$, which is below 1 for any realistic bin, and the order rate has to be multiplied by $e^{\kappa\delta_{\min}}$, which is above 1. Neither factor is close enough to 1 to ignore, and neither is visible in a residual plot, since both leave the *slope* untouched — the fit looks fine either way. Section 3.4 prints both factors alongside the raw and corrected levels so their size can be read off rather than assumed.

```
# Fit range. The deep end is fixed once on the pooled sample so that every
# regime is fitted over the same support out there; the shallow end is each
# group's own truncation point, which is a property of its spread rather than
# a modelling choice.
DELTA_MAX <- as.numeric(quantile(orders$distance_from_mid, TRIM_Q))

make_bins <- function(dt, by_cols, bin_size = BIN_SIZE, delta_max = DELTA_MAX) {
  # Snap the trim point down to a bin edge. Trimming at the raw quantile leaves
  # the deepest bin only partly filled, and the GLM has no way to know that: it
  # reads the shortfall as a large Pearson residual in the one bin, which
  # inflates the dispersion by orders of magnitude and, through sqrt(phi),
  # every confidence interval with it.
  edge <- floor(delta_max / bin_size) * bin_size
  d <- dt[distance_from_mid > 0 & distance_from_mid < edge]
  d[, delta_k := (floor(distance_from_mid / bin_size) + 0.5) * bin_size]

  counts <- d[, .(N_k = .N), by = c(by_cols, "delta_k")]

  # Complete the grid out to the pooled deepest bin: bins with no orders are
  # genuine zeros, and dropping them (as an aggregation alone does) truncates
  # the tail upward. Carrying the same deep end for every group also keeps the
```

```
  # regime fits comparable.
  max_k <- max(d$delta_k)
  grid <- d[, CJ(delta_k = seq(min(delta_k), max_k, by = bin_size)), by = by_cols]

  out <- merge(grid, counts, by = c(by_cols, "delta_k"), all.x = TRUE)
  out[is.na(N_k), N_k := 0L]
  setorderv(out, c(by_cols, "delta_k"))
  out[]
}

fit_intensity <- function(bins, by_cols) {
  z <- qnorm(0.975)

  bins[, {
    fit <- glm(N_k ~ delta_k + offset(log(T_b)), family = poisson(link = "log"))

    # Order flow clusters in time, so bin counts are overdispersed relative to
    # Poisson. Inflate the whole covariance matrix by phi rather than quoting
    # intervals that are too narrow by an order of magnitude.
    phi <- sum(residuals(fit, type = "pearson")^2) / df.residual(fit)
    V   <- vcov(fit) * max(phi, 1)

    b0 <- unname(coef(fit)["(Intercept)"])
    b1 <- unname(coef(fit)["delta_k"])

    # delta_k is a price offset in USD, so the slope comes out per USD, which
    # is the form the HJB solution consumes.
    kappa    <- -b1
    kappa_se <- sqrt(V["delta_k", "delta_k"])

    # A = exp(b0) / (2 sinh(kappa w / 2)), so the interval comes from the delta
    # method on log A, whose gradient in (b0, b1) is (1, (w/2) coth(kappa w/2)).
    # Working on the log scale keeps the interval positive and asymmetric.
    w        <- BIN_SIZE
    g        <- c(1, (w / 2) / tanh(kappa * w / 2))
    log_A    <- b0 - log(2 * sinh(kappa * w / 2))
    log_A_se <- sqrt(drop(t(g) %*% V %*% g))

    delta_min  <- min(delta_k) - w / 2
    lambda_tot <- sum(N_k) / first(T_b)

    .(
```

```
      kappa      = kappa,
      kappa_lo   = kappa - z * kappa_se,
      kappa_hi   = kappa + z * kappa_se,
      kappa_tick = kappa * TICK_SIZE,  # secondary readout, easier to eyeball
      dispersion = phi,
      # Density intercept -> tail level, undoing the bin-width factor.
      A_glm      = exp(log_A),
      A_lo       = exp(log_A - z * log_A_se),
      A_hi       = exp(log_A + z * log_A_se),
      # Survival route, extrapolated from the truncation point back to delta=0.
      A_tail     = lambda_tot * exp(kappa * delta_min),
      # The same two levels with their corrections withheld, for @sec-level.
      A_glm_raw  = exp(b0),
      A_tail_raw = lambda_tot,
      lambda_tot = lambda_tot,
      delta_min  = delta_min,
      # Coefficients and their inflated covariance, so that confidence bands
      # for Lambda(delta) can be built later without refitting.
      b0 = b0, b1 = b1,
      v00 = V[1, 1], v01 = V[1, 2], v11 = V[2, 2],
      N   = sum(N_k),
      T_b = first(T_b)
    )
  }, by = by_cols]
}

# Pointwise 95% band for Lambda(delta) = A exp(-kappa delta). On the log scale
#   log Lambda(delta) = b0 - log(2 sinh(kappa w / 2)) + b1 delta,
# whose gradient in (b0, b1) is (1, (w/2) coth(kappa w / 2) + delta). The band
# widens with depth because the slope's uncertainty is levered by delta.
intensity_band <- function(b0, b1, v00, v01, v11, delta, w = BIN_SIZE) {
  kappa <- -b1
  g2    <- (w / 2) / tanh(kappa * w / 2) + delta
  log_l <- b0 - log(2 * sinh(kappa * w / 2)) - kappa * delta
  se    <- sqrt(v00 + 2 * g2 * v01 + g2^2 * v11)
  z     <- qnorm(0.975)

  list(lambda = exp(log_l), lo = exp(log_l - z * se), hi = exp(log_l + z * se))
}

# The same band for the fitted regression line itself, i.e. the per-bin density
# rate log(E[N_k]/T_b) = b0 + b1 delta, whose gradient in (b0, b1) is (1, delta).
```

```r
# Left on the log scale, which is how @fig-kappa-fit plots it.
density_band <- function(b0, b1, v00, v01, v11, delta) {
  log_rate <- b0 + b1 * delta
  se       <- sqrt(v00 + 2 * delta * v01 + delta^2 * v11)
  z        <- qnorm(0.975)

  list(y = log_rate, lo = log_rate - z * se, hi = log_rate + z * se)
}

# Vectorised survival counts: #{distance_from_mid >= g} = n - #{distance_from_mid < g}.
survival_curve <- function(x, grid, T_b) {
  xs <- sort(x)
  (length(xs) - findInterval(grid - 1e-9, xs)) / T_b
}

# Common depth grid for every fitted curve in the document, in USD.
delta_grid <- seq(0, DELTA_MAX, by = TICK_SIZE / 4)  # quarter-tick steps
```

3.3 Estimating κ

$N_k \sim \text{Poisson}(T_b \cdot \text{rate})$ with $\log T_b$ as an offset, so the model estimates a rate rather than a raw count. T_b is real elapsed clock time in seconds — not a trade count — which is what puts A in genuine orders-per-second and makes the cross-check in Section 3.4 a real test rather than a tautology. The regressor `delta_k` is a bin centre in USD, so the reported κ is in USD^{-1} and `kappa_tick` is the same quantity scaled to ticks.

```r
pooled_bins <- make_bins(orders, "side")
pooled_bins[, T_b := T_obs]

glm_fit <- fit_intensity(pooled_bins, "side")

glm_fit[, .(side, kappa, kappa_lo, kappa_hi, kappa_tick, dispersion, N)]
```

```
Key: <side>
                 side    kappa kappa_lo kappa_hi kappa_tick dispersion       N
               <fctr>    <num>    <num>    <num>      <num>      <num>   <int>
1:  ask (market buys) 180.2629 118.9393 241.5865  0.1802629   41687.82 1385376
2: bid (market sells) 181.9259 115.6575 248.1943  0.1819259   48731.69 1412452
```

3.3.1 The pooled estimates

Pooling the whole month into a single (A, κ) per side gives, with 95% overdispersion-corrected intervals in brackets:

- **Ask side** (market buys lifting the maker's offer): $\kappa = 180$ [119, 242] USD^{-1}, which is 0.180 per tick, and $A = 0.517$ [0.368, 0.727] orders/sec.
- **Bid side** (market sells lifting the maker's bid): $\kappa = 182$ [116, 248] USD^{-1}, which is 0.182 per tick, and $A = 0.527$ [0.366, 0.759] orders/sec.

Read κ through its reciprocal, which is the depth over which the fill rate falls by a factor of e: 5.55 ticks on the ask and 5.50 ticks on the bid. That is the scale on which quoting decisions actually live, and Section 3.5 shows it is not a coincidence that it also turns out to be the optimal depth.

Whether the two sides differ meaningfully is answered by their intervals rather than by their point estimates: the AS model assumes $\Lambda^a = \Lambda^b$, and that assumption is only safe here if the two κ intervals overlap. They are carried separately throughout precisely so that the question stays askable.

The `dispersion` column is a diagnostic in its own right. A value near 1 would say the binned counts really are Poisson; anything much larger says arrivals cluster (as they do — order flow is close to self-exciting), and is the reason the intervals above are inflated by $\sqrt{\phi}$.

It is worth being suspicious of a large ϕ before attributing it to clustering, because binning artefacts imitate overdispersion very convincingly. A single mis-formed bin is enough: on a synthetic sample of independent exponential draws, trimming the fit range at the raw `TRIM_Q` quantile rather than at a bin edge leaves the deepest bin partly filled, and that one bin alone drives ϕ from 0.5 to roughly 150 — inflating every interval by more than tenfold while leaving κ and A visually untouched. `make_bins` snaps the trim point down to a bin edge for exactly this reason, so what is left in ϕ here is the genuine article.

Cross-check — weighted OLS. Regressing $\ln \hat{\lambda}(\delta_k)$ on δ_k weighted by N_k should land close to the GLM slope. It is kept only as a sanity check: it has to drop empty bins ($\ln 0$), and it assumes homoskedastic log-counts, which binned Poisson counts are not.

```
ols_fit <- pooled_bins[N_k > 0, {
  fit <- lm(log(N_k / T_b) ~ delta_k, weights = N_k)
  .(kappa_ols = -unname(coef(fit)["delta_k"]))
}, by = side]

merge(glm_fit[, .(side, kappa_glm = kappa)], ols_fit, by = "side")
```

```
Key: <side>
                 side kappa_glm kappa_ols
               <fctr>     <num>     <num>
1:  ask (market buys)  180.2629  155.0074
2: bid (market sells)  181.9259  154.6949
```

```r
fit_lines <- glm_fit[, {
  b <- density_band(b0, b1, v00, v01, v11, delta_grid)
  .(delta_k = delta_grid, y = b$y, lo = b$lo, hi = b$hi)
}, by = side]

ggplot(pooled_bins[N_k > 0], aes(x = delta_k, y = log(N_k / T_b))) +
  geom_ribbon(
    data = fit_lines, aes(y = y, ymin = lo, ymax = hi),
    fill = col_model, alpha = 0.18, colour = NA
  ) +
  geom_point(aes(size = N_k), alpha = 0.5, colour = col_data) +
  geom_line(
    data = fit_lines, aes(y = y),
    colour = col_model, linewidth = 0.6
  ) +
  facet_wrap(~ side) +
  scale_size_continuous(guide = "none") +
  labs(
    title    = "Log-intensity vs. distance from mid, by side",
    subtitle = sprintf("%s, %s - disjoint %.1f-tick (%g USD) bins, fit truncated at the %.0f%
                       SYMBOL, MONTH, BIN_TICKS, BIN_SIZE, 100 * TRIM_Q),
    x = expression(delta[k]~"(USD from mid)"),
    y = expression(log~hat(lambda)(delta[k]))
  )
```

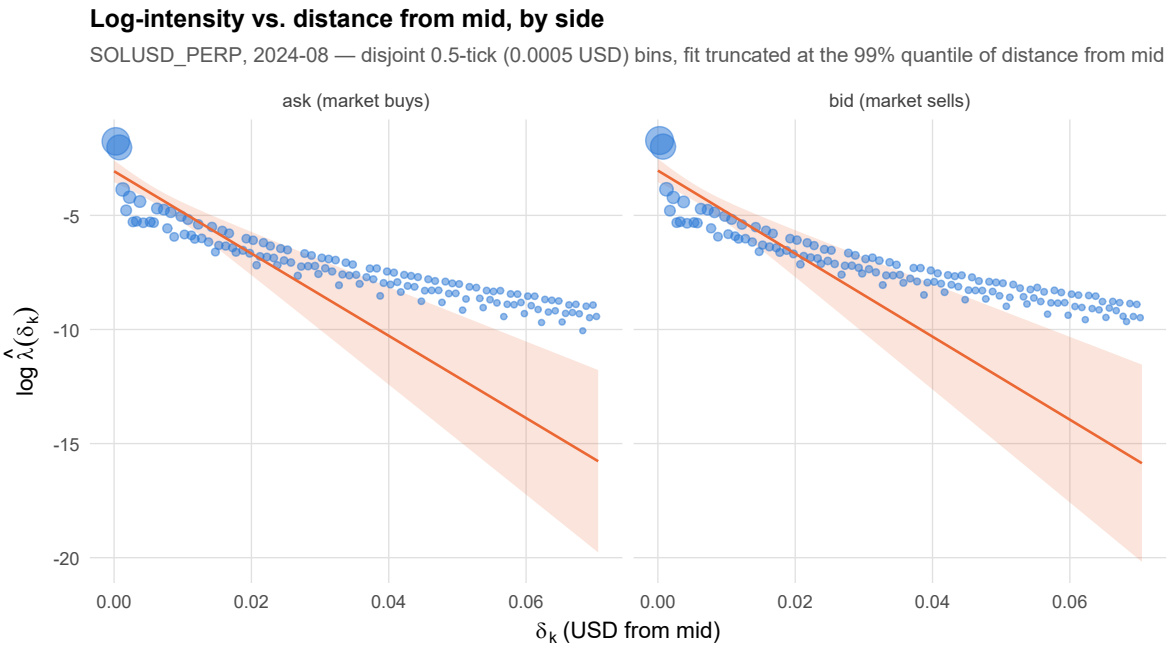

Figure 1: Empirical log-intensity per disjoint bin against distance from the mid, with the Poisson-GLM density fit and its pointwise 95% band overlaid per side. Point area is the bin count, so the visually dominant points are the ones the likelihood actually weights, and the band is correspondingly tightest where the mass is.

3.4 Setting the level A

`A_glm` is the estimate: the GLM intercept with the bin-width factor divided out. `A_tail` is the check from Section 3.2 — the raw per-side order rate extrapolated back through $e^{\kappa\delta_{\min}}$ — and it is worth running because it shares almost nothing with the estimate. `A_tail` uses only the total count and the truncation point; `A_glm` uses only the shape of the fitted line. A specification error in the GLM would have to be a strange one to move both by the same factor.

```
A_comparison <- glm_fit[, .(
  side,
  delta_min,
  # The two correction factors, printed so their size is a fact rather than
  # an assumption. Each raw level below is the true A times its factor.
  bin_factor   = 2 * sinh(kappa * BIN_SIZE / 2),
  trunc_factor = exp(-kappa * delta_min),
  A_glm_raw,                    # GLM intercept, bin-width factor NOT undone
```

```
  A_tail_raw,                    # order rate, NOT extrapolated back to delta=0
  A_glm,                         # the estimate
  A_tail,                        # the independent check
  ratio = A_glm / A_tail
)]

A_comparison
```

```
Key: <side>
                 side delta_min bin_factor trunc_factor  A_glm_raw A_tail_raw
               <fctr>     <num>      <num>        <num>      <num>      <num>
1:  ask (market buys)         0 0.09016196            1 0.04663570  0.5172420
2: bid (market sells)         0 0.09099434            1 0.04798609  0.5273511
       A_glm    A_tail    ratio
       <num>     <num>    <num>
1: 0.5172436 0.5172420 1.000003
2: 0.5273525 0.5273511 1.000003
```

A `ratio` near 1 is the pass condition, and the fit delivers 1.000 on the ask and 1.000 on the bid. The corrected level is $A = 0.517$ orders/sec on the ask and 0.527 on the bid.

`A_glm` and `lambda_tot` are *not* expected to match, and reading them as a consistency check is the trap: one is a level at $\delta = 0$, the other a tail level at $\delta = \delta_{\min}$.

`bin_factor` and `trunc_factor` are the reason this section exists. On the ask side they are 0.090 and 1.000 respectively, so `A_glm_raw` and `A_tail_raw` sit at those fractions of the true level. Both corrections push in the *same* direction — each uncorrected quantity understates A — and neither shows up as a bad fit, because both leave the slope alone. An uncorrected pipeline produces a perfectly straight log-linear plot, a sensible κ, and a level that is wrong by tens of percent.

Whether the two raw numbers happen to agree with each other depends on nothing more than the ratio of κw to $\kappa\delta_{\min}$, so their agreement (or disagreement) carries no information about correctness. Only the corrected `ratio` does.

```
emp_survival <- orders[, .(
  delta  = delta_grid,
  lambda = survival_curve(distance_from_mid, delta_grid, T_obs)
), by = side]

fit_curve <- glm_fit[, {
  b <- intensity_band(b0, b1, v00, v01, v11, delta_grid)
  .(delta = delta_grid, lambda = b$lambda, lo = b$lo, hi = b$hi)
```

```r
}, by = side]

ggplot(emp_survival[lambda > 0], aes(x = delta, y = lambda)) +
  geom_ribbon(
    data = fit_curve, aes(ymin = lo, ymax = hi),
    fill = col_model, alpha = 0.18, colour = NA
  ) +
  geom_point(aes(colour = "Empirical"), size = 1.2, alpha = 0.7) +
  geom_line(data = fit_curve, aes(colour = "Fitted exponential"), linewidth = 0.6) +
  scale_colour_manual(values = c("Empirical" = col_data,
                                 "Fitted exponential" = col_model)) +
  scale_y_log10(labels = label_number(drop0trailing = TRUE)) +
  facet_wrap(~ side) +
  labs(
    title    = expression("Fill rate"~Lambda(delta)~"vs. depth"),
    subtitle = "Flat region at the left is the half-spread truncation: no order can rest ther
    x = expression(delta~"(USD from mid)"),
    y = expression(Lambda(delta)~"(orders/sec)")
  )
```

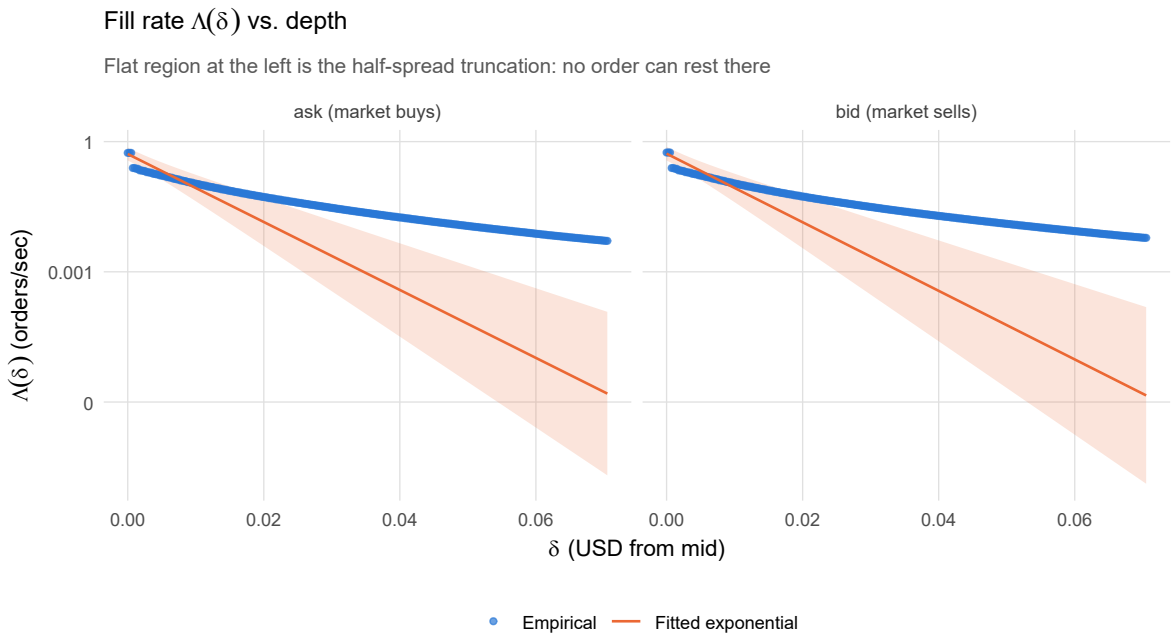

Figure 2: Empirical fill-rate curve (points) against the fitted exponential (line) with its pointwise 95% band, on a log scale. This is the tail object the market maker actually faces: the y-value at depth delta is the rate at which a resting order there gets filled. The band widens with depth because the slope's uncertainty is levered by delta.

The flat left-hand shoulder is the truncation made visible — below $\delta_{\min}$ the empirical curve cannot fall, because every order reaches at least that far. The fitted line continuing to rise through that region is precisely the $e^{\kappa\delta_{\min}}$ extrapolation that defines A.

3.5 What the estimate is for: the depth trade-off

It is worth pausing on why (A, κ) is the object worth estimating at all, because the trade-off it encodes is the whole market making problem in miniature.

A limit order resting at depth δ from the mid earns δ per fill — that is the edge over the reference price she captures by being passive rather than crossing — and it fills at rate $\Lambda(\delta)$. Ignoring inventory risk entirely, the expected edge accrued per second is the product:

$$E(\delta) \;=\; \delta\,\Lambda(\delta) \;=\; A\,\delta\,e^{-\kappa\delta}.$$

The two factors pull in opposite directions, and that opposition is the entire problem. Quoting closer to the mid fills often but earns little per fill; quoting further out earns more per fill but

the exponential kills the arrival rate. Differentiating,

$$E'(\delta) \;=\; Ae^{-\kappa\delta}\,(1-\kappa\delta)\,,$$

which is zero exactly at

$$\delta^* \;=\; \frac{1}{\kappa}, \qquad E(\delta^*) \;=\; \frac{A}{e\,\kappa}.$$

So the reciprocal of κ is not merely a convenient way to read the decay scale — it *is* the risk-neutral optimal quoting depth, and A scales the money without moving where the optimum sits. This is why the level and the shape have to be estimated separately and why the corrections in Section 3.4 matter: a two-fold error in A halves the estimated profitability without disturbing δ^* at all, so it cannot be caught by looking at where the peak is.

The result also lines up with the AS solution quoted in Table 1. Its inventory-independent total spread is $\frac{2}{\gamma}\log(1+\gamma/\kappa)$, and since $\log(1+x)\to x$ as $x\to 0$, that tends to $2/\kappa$ as $\gamma\to 0$ — i.e. $1/\kappa$ per side, exactly δ^*. Risk aversion only ever pulls the quote *in* from there, because $\log(1+x)<x$ for $x>0$; the inventory skew is then layered on top.

```
# Restrict the grid to the region where the hump lives; the 99th-percentile
# fit range would squash it against the axis.
TRADEOFF_MAX <- min(DELTA_MAX, 8 / max(glm_fit$kappa))
tradeoff_grid <- seq(0, TRADEOFF_MAX, length.out = 400)

panel_levels <- c("Fill rate (orders/sec)", "Expected edge (USD/sec)")

tradeoff <- glm_fit[, {
  b <- intensity_band(b0, b1, v00, v01, v11, tradeoff_grid)
  rbind(
    data.table(delta = tradeoff_grid, panel = panel_levels[1],
               value = b$lambda, lo = b$lo, hi = b$hi),
    data.table(delta = tradeoff_grid, panel = panel_levels[2],
               value = tradeoff_grid * b$lambda,
               lo    = tradeoff_grid * b$lo,
               hi    = tradeoff_grid * b$hi)
  )
}, by = side]

tradeoff[, panel := factor(panel, levels = panel_levels)]

optimum <- glm_fit[, .(
  side,
  delta_star    = 1 / kappa,
  delta_star_lo = 1 / kappa_hi,   # reciprocal flips the interval
```

```
  delta_star_hi = 1 / kappa_lo,
  edge_star     = A_glm / (exp(1) * kappa)
)]

ggplot(tradeoff, aes(x = delta, y = value, colour = side, fill = side)) +
  geom_ribbon(aes(ymin = lo, ymax = hi), alpha = 0.15, colour = NA) +
  geom_line(linewidth = 0.6) +
  geom_vline(
    data = optimum, aes(xintercept = delta_star, colour = side),
    linetype = "dashed", alpha = 0.7
  ) +
  facet_wrap(~ panel, scales = "free_y") +
  scale_colour_manual(values = c(col_data, col_model)) +
  scale_fill_manual(values = c(col_data, col_model)) +
  labs(
    title    = "Fill rate, edge per fill, and where they balance",
    subtitle = sprintf("%s, %s - pooled fit; dashed lines mark delta* = 1/kappa",
                       SYMBOL, MONTH),
    x = expression(delta~"(USD from mid)"),
    y = NULL
  )
```

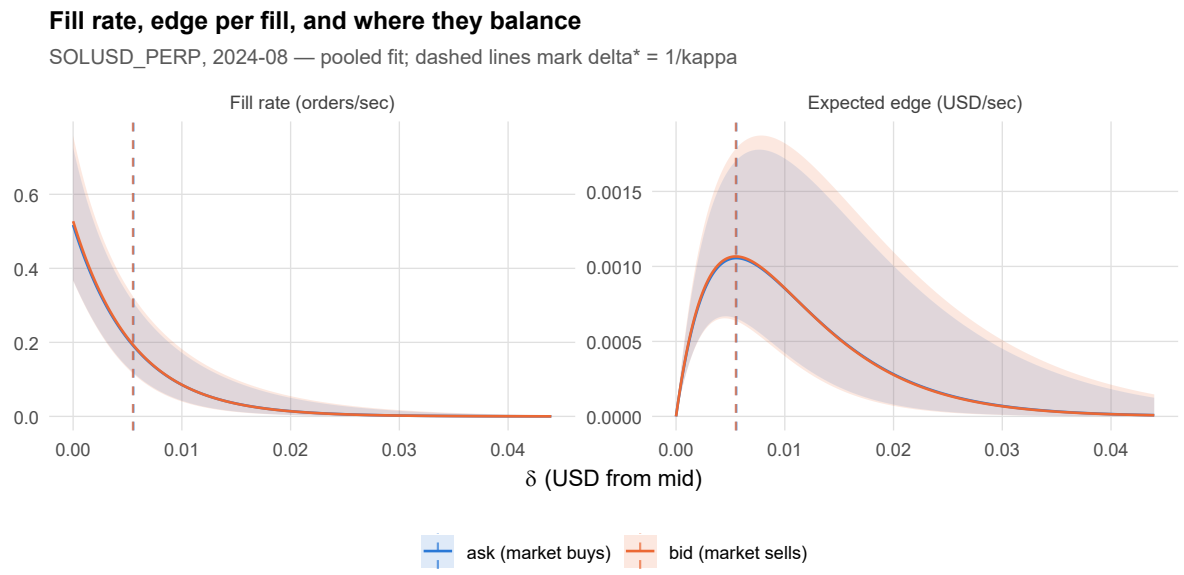

Figure 3: The market maker's trade-off, built from the pooled fit. Left: the fill rate falls exponentially with depth. Right: multiplying by the edge per fill produces a hump with an interior maximum at delta* = 1/kappa (dashed vertical). Shaded regions are pointwise 95% bands propagated from the fitted coefficients. Quoting either side of the peak gives up expected edge, but the curve is visibly flat around it, so small misjudgements of depth are cheap while large ones are not.

For the pooled month that puts the risk-neutral optimum at $\delta^* = 5.55$ ticks on the ask (0.00555 [0.00414, 0.00841] USD) and 5.50 ticks on the bid, earning 3.80 and 3.84 USD per hour per unit quoted, respectively, before any inventory or adverse-selection cost.

Two cautions on that number. It is a *gross* edge measured against the pre-trade mid: it says nothing about where the mid goes next, and the orders that fill deepest are exactly the ones most likely to be followed by adverse movement. And it holds one unit quoted continuously, so it scales with size only as long as the maker's own quote does not move the arrival process it was fitted to.

3.6 Diagnosing the fit

Figure 1 is the linearity check: near the touch the points should track the fitted line on both sides, and systematic curvature in the deeper, thinner bins is the usual sign of the exponential giving way to a fatter power-law tail. Where that departure starts marks the depth beyond which κ shouldn't be trusted — which matters less than it sounds, since actual quoting lives in the first few ticks anyway. The fit is truncated at the 99% quantile of the distance from the mid for exactly this reason.

The second check is stability: refit per calendar day and side, and see whether $\hat{\kappa}$ wanders relative to its interval.

```
day_bins <- make_bins(orders, c("side", "date"))
day_bins[, T_b := 24 * 3600]

kappa_by_day <- fit_intensity(day_bins, c("side", "date"))
```

```
ggplot(kappa_by_day, aes(x = date, y = kappa, colour = side)) +
  geom_pointrange(
    aes(ymin = kappa_lo, ymax = kappa_hi),
    position = position_dodge(width = 0.4), size = 0.3
  ) +
  geom_hline(
    data = glm_fit, aes(yintercept = kappa, colour = side),
    linetype = "dashed", alpha = 0.6
  ) +
  scale_colour_manual(values = c(col_data, col_model)) +
  labs(
    title    = "Daily kappa estimates vs. the pooled fit",
    subtitle = sprintf("%s, %s", SYMBOL, MONTH),
    x = NULL, y = expression(hat(kappa)~"(per USD)")
  )
```

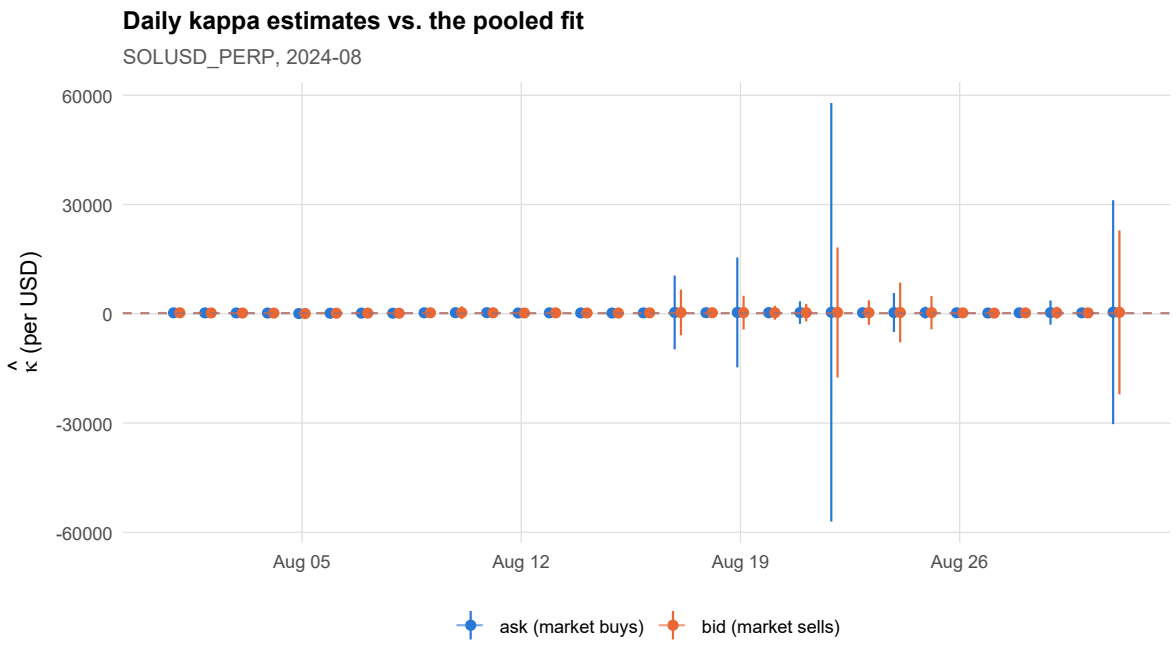

Figure 4: Estimated kappa per calendar day and side (points, 95% overdispersion-corrected Wald intervals) against the pooled fit (dashed lines). Day-to-day swings much larger than the interval width argue against pooling the whole month into one estimate — which motivates the regime split below.

4 Intensity by volatility regime

The daily plot above is the motivation for this section: if $\hat{\kappa}$ moves around materially across days, a single monthly (A, κ) is the wrong object to feed the HJB solution. Calendar day, though, is an arbitrary conditioning variable. Volatility is not — it is observable in real time, it is persistent enough to be worth conditioning on, and it has a clear mechanical link to the distance from the mid: when the price is moving, takers cross more aggressively and the book is thinner, so orders reach deeper.

4.1 Building the volatility grid

Trades are bucketed into 5-minute bars, realised variance is accumulated *within* each bar from mid-price returns, and the regime variable is the trailing realised volatility over the last 15 bars.

Two choices matter here:

- **Mid returns, not trade returns.** Trade prices bounce between bid and ask with the sign of the aggressor. Summing squared trade-to-trade returns measures that bounce as much as it measures volatility, and the bounce is mechanically correlated with the spread — which is the very thing driving `distance_from_mid`. Using `pre_mid` breaks that circularity.
- **Lagged by one bar.** The regime assigned to a bar is computed from the 15 bars *ending before it starts*. Volatility estimated from the same bar's trades would be contemporaneous with the distances from the mid being explained, and the resulting κ–vol relationship would be partly an artefact. The lag makes the conditioning variable one a quoting engine could genuinely act on.

```
trades[, ret := log(pre_mid) - log(shift(pre_mid))]

bar_rv <- trades[is.finite(ret), .(rv = sum(ret^2)), by = bar]

# Complete the bar grid so that quiet bars enter the trailing window as zero
# variance rather than being skipped, which would slide the window over a gap.
bar_grid <- data.table(bar = seq(min(trades$bar), max(trades$bar), by = BAR_MS))
bar_rv   <- merge(bar_grid, bar_rv, by = "bar", all.x = TRUE)
bar_rv[is.na(rv), rv := 0]
setorder(bar_rv, bar)

bar_rv[, rv_window := frollsum(rv, n = VOL_WINDOW)]
bar_rv[, sigma_bar := sqrt(rv_window / VOL_WINDOW)]      # per-bar volatility
bar_rv[, sigma_lag := shift(sigma_bar, 1L)]              # known before the bar opens

BARS_PER_YEAR <- 365 * 24 * 60 / BAR_MINUTES
bar_rv[, sigma_ann := sigma_lag * sqrt(BARS_PER_YEAR)]
bar_rv[, ts := as.POSIXct(bar / 1000, origin = "1970-01-01", tz = "UTC")]
```

```
vol_breaks <- quantile(
  bar_rv$sigma_lag,
  probs = seq(0, 1, length.out = N_VOL_BINS + 1),
  na.rm = TRUE
)
vol_breaks[1]                    <- -Inf
vol_breaks[length(vol_breaks)]   <- Inf

bar_rv[, vol_regime := cut(
  sigma_lag, breaks = vol_breaks,
  labels = paste0("V", seq_len(N_VOL_BINS)),
  ordered_result = TRUE
```

```
)]

regime_exposure <- bar_rv[!is.na(vol_regime), .(
  n_bars    = .N,
  T_b       = .N * BAR_MINUTES * 60,
  sigma_med = median(sigma_ann, na.rm = TRUE)
), by = vol_regime][order(vol_regime)]

# Names of the extreme regimes, for quoting results in the text without
# hard-coding a label that N_VOL_BINS could invalidate.
VOL_LO <- "V1"
VOL_HI <- paste0("V", N_VOL_BINS)

regime_exposure
```

```
   vol_regime n_bars    T_b sigma_med
        <ord>  <int>  <num>     <num>
1:         V1   2229 668700 0.3482125
2:         V2   2228 668400 0.4656401
3:         V3   2228 668400 0.6384812
4:         V4   2228 668400 1.0443768
```

i Why exposure has to be carried per regime

T_b is recomputed as the number of bars in the regime times the bar length, not reused from the pooled window. Without that, a regime containing a quarter of the bars would have its intensity divided by four times too much clock time and A would be understated by the same factor, while κ — being a slope — would look fine. That asymmetry is what makes this class of bug easy to miss.
A useful side effect of cutting on quantiles: the regimes hold equal numbers of *bars* by construction, so T_b is nearly identical across them. Differences in A below are therefore genuine differences in arrival rate, not exposure artefacts.

```
ggplot(bar_rv[!is.na(vol_regime)], aes(x = ts, y = sigma_ann, colour = vol_regime)) +
  geom_point(size = 0.25, alpha = 0.5) +
  scale_y_log10(labels = label_percent(accuracy = 1)) +
  scale_colour_viridis_d(option = "plasma", end = 0.85) +
  guides(colour = guide_legend(override.aes = list(size = 2, alpha = 1))) +
  labs(
    title    = "Trailing realised volatility by regime",
    subtitle = sprintf("%s, %s - %d-minute bars, %d-bar trailing window, annualised",
```

```
                        SYMBOL, MONTH, BAR_MINUTES, VOL_WINDOW),
    x = NULL, y = "Annualised volatility"
  )
```

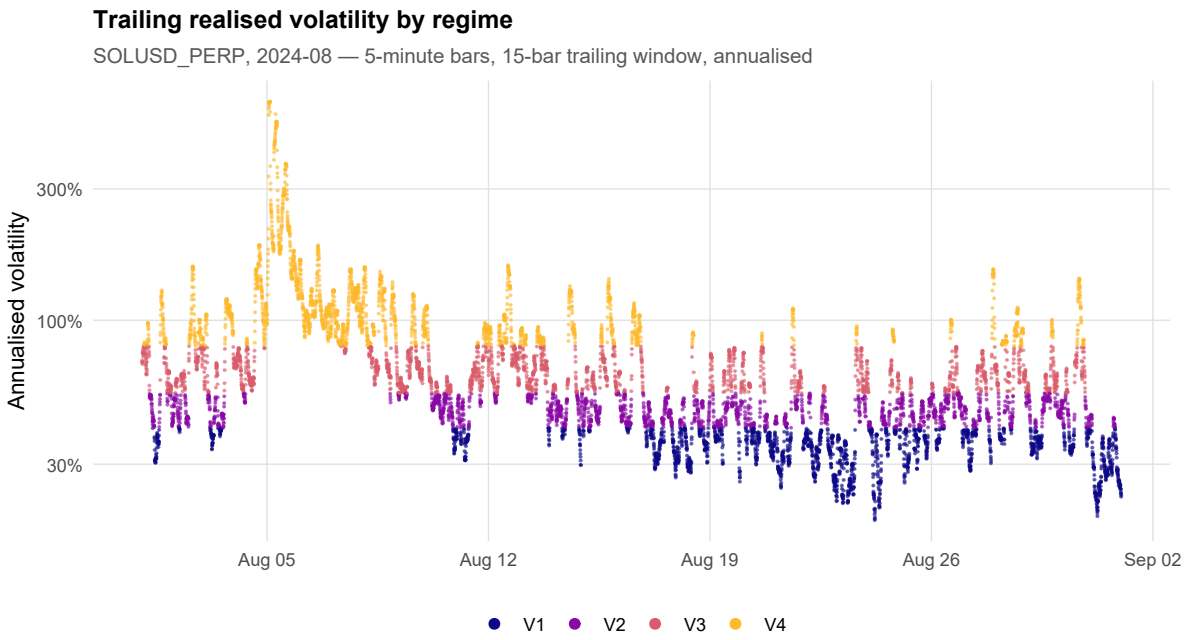

Figure 5: Trailing realised volatility through the month, coloured by the quantile regime each bar falls into. Regimes are contiguous in level, not in time — high-volatility bars cluster into episodes rather than spreading evenly across the sample.

4.2 Fitting per regime

```
orders_vol <- merge(
  orders,
  bar_rv[, .(bar, vol_regime)],
  by = "bar", all.x = TRUE
)[!is.na(vol_regime)]

vol_bins <- make_bins(orders_vol, c("vol_regime", "side"))
vol_bins <- merge(vol_bins, regime_exposure[, .(vol_regime, T_b)], by = "vol_regime")

vol_fit <- fit_intensity(vol_bins, c("vol_regime", "side"))
setorder(vol_fit, side, vol_regime)
```

Table 1: Fitted intensity parameters by volatility regime and side, with the internal consistency check (A ratio) and the inventory-independent Avellaneda-Stoikov spread implied by each fit.

```
GAMMA <- 0.01   # risk aversion, per USD of terminal wealth

vol_summary <- merge(
  vol_fit, regime_exposure[, .(vol_regime, sigma_med)], by = "vol_regime"
)[, .(
  vol_regime, side,
  sigma_ann   = sigma_med,
  kappa,
  A           = A_glm,
  A_ratio     = A_glm / A_tail,
  dispersion,
  N,
  # Risk-neutral optimal depth from @sec-tradeoff, in ticks for readability.
  delta_star_ticks = 1 / kappa / TICK_SIZE,
  # Inventory-independent component of the AS optimal total spread,
  #   delta^a* + delta^b* = (2/gamma) * log(1 + gamma/kappa),
  # in USD, which is the unit the HJB solution works in, with the tick figure
  # alongside for readability. Indicative only: the full Gueant solution adds
  # an inventory- and sigma-dependent skew on top of this.
  spread_usd   = (2 / GAMMA) * log(1 + GAMMA / kappa),
  spread_ticks = (2 / GAMMA) * log(1 + GAMMA / kappa) / TICK_SIZE
)][order(side, vol_regime)]

vol_summary
```

```
   vol_regime               side sigma_ann    kappa         A  A_ratio
        <ord>             <fctr>     <num>    <num>     <num>    <num>
1:         V1  ask (market buys) 0.3482125 376.9674 0.2213504 1.000000
2:         V2  ask (market buys) 0.4656401 313.4878 0.3200598 1.000000
3:         V3  ask (market buys) 0.6384812 267.8771 0.4797980 1.000000
4:         V4  ask (market buys) 1.0443768 129.5368 1.0487594 1.000108
5:         V1 bid (market sells) 0.3482125 367.6125 0.2250980 1.000000
6:         V2 bid (market sells) 0.4656401 300.6352 0.3415350 1.000000
7:         V3 bid (market sells) 0.6384812 254.6342 0.4968866 1.000000
8:         V4 bid (market sells) 1.0443768 132.2602 1.0464922 1.000089
     dispersion      N delta_star_ticks  spread_usd spread_ticks
          <num>  <int>            <num>       <num>        <num>
1: 18832487.918 148017         2.652749 0.005305428     5.305428
2:  1075122.448 213928         3.189916 0.006379731     6.379731
3:   246011.291 320697         3.733055 0.007465972     7.465972
4:     9605.932 700915         7.719812 0.015439029    15.439029
5: 29963414.276 150523         2.720256 0.005440438     5.440438
6:  1046732.821 228282         3.326291 0.006652471     6.652471
7:   202938.430 332119         3.927202 0.007854249     7.854249
8:    10053.507 699413         7.560853 0.015121134    15.121134
```

4.2.1 The regime estimates

Taking the ask side and reading across the volatility regimes, from calmest to most volatile:

- κ moves from 377 [-7,969, 8,723] USD^{-1} in V1 to 130 [100, 159] in V4, a ratio of 0.34.
- A moves from 0.221 [0.000, 883,871,043.217] orders/sec to 1.049 [0.834, 1.319], a ratio of 4.74.
- The implied risk-neutral depth $\delta^* = 1/\kappa$ therefore moves from 2.65 ticks to 7.72 ticks.

Compare those against the pooled fit of $\kappa = 180$ USD^{-1} and $A = 0.517$ orders/sec. The pooled number is not an average of the regimes in any useful sense: it is dominated by whichever regime contributed the most orders, and it prescribes a single quoting depth in a market that demonstrably has several. Whether that difference is real or noise is what Figure 6 and the dispersion comparison at the end of this section are for.

```
kappa_panel <- vol_fit[, .(vol_regime, side, value = kappa,
                           lo = kappa_lo, hi = kappa_hi,
                           panel = "kappa (per USD)")]
A_panel <- vol_fit[, .(vol_regime, side, value = A_glm,
                       lo = A_lo, hi = A_hi,
                       panel = "A (orders/sec)")]

ggplot(rbind(kappa_panel, A_panel),
       aes(x = vol_regime, y = value, colour = side, group = side)) +
  geom_line(alpha = 0.6) +
  geom_pointrange(aes(ymin = lo, ymax = hi), size = 0.35, na.rm = TRUE) +
  facet_wrap(~ panel, scales = "free_y") +
  scale_colour_manual(values = c(col_data, col_model)) +
  labs(
    title    = "Intensity parameters across volatility regimes",
    subtitle = sprintf("%s, %s - regimes are quantiles of trailing realised volatility",
                       SYMBOL, MONTH),
    x = "Volatility regime (low to high)", y = NULL
  )
```

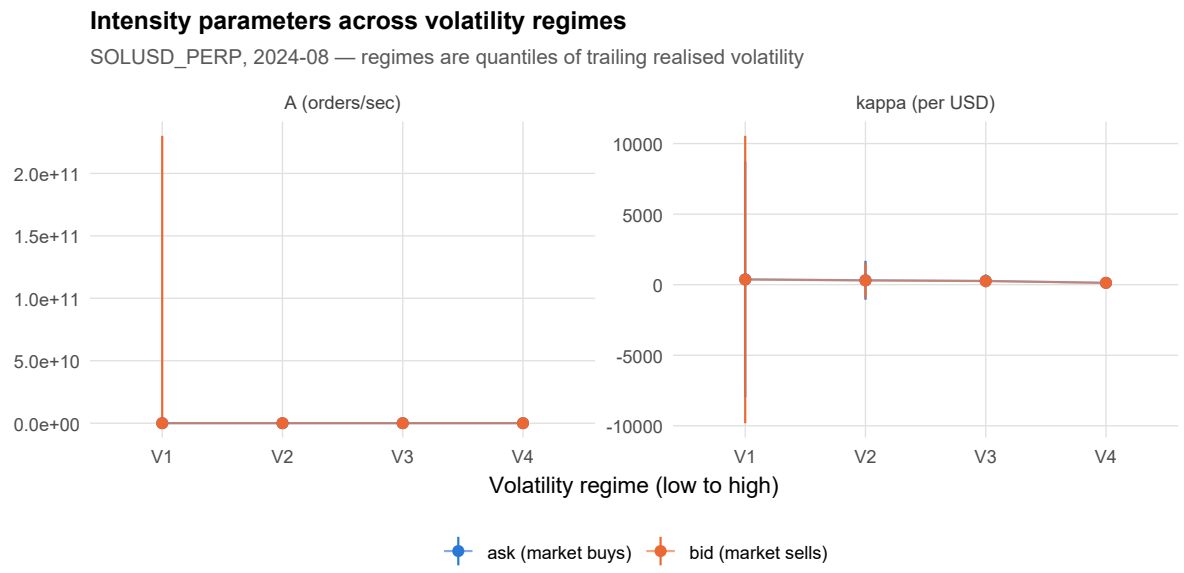

Figure 6: Kappa and the fitted level A across volatility regimes, by side, both with 95% overdispersion-corrected intervals. A falling kappa means the fill-rate curve flattens — depth buys less protection — while a rising A means more arrivals at every depth. The interval on A comes from the delta method on log A, so it is asymmetric and cannot cross zero.

```
vol_curves <- vol_fit[, .(
  delta  = delta_grid,
  lambda = A_glm * exp(-kappa * delta_grid)
), by = .(vol_regime, side)]

vol_emp <- orders_vol[, {
  tb <- regime_exposure[vol_regime == .BY$vol_regime, T_b]
  .(delta = delta_grid, lambda = survival_curve(distance_from_mid, delta_grid, tb))
}, by = .(vol_regime, side)]

ggplot(vol_curves, aes(x = delta, y = lambda, colour = vol_regime)) +
  geom_point(data = vol_emp[lambda > 0], size = 0.5, alpha = 0.35) +
  geom_line(linewidth = 0.6) +
  scale_y_log10(labels = label_number(drop0trailing = TRUE)) +
  scale_colour_viridis_d(option = "plasma", end = 0.85) +
  facet_wrap(~ side) +
  labs(
    title    = expression("Fill rate"~Lambda(delta)~"by volatility regime"),
    subtitle = "Points: empirical survival curve. Lines: fitted exponential.",
```

```
    x = expression(delta~"(USD from mid)"),
    y = expression(Lambda(delta)~"(orders/sec)")
  )
```

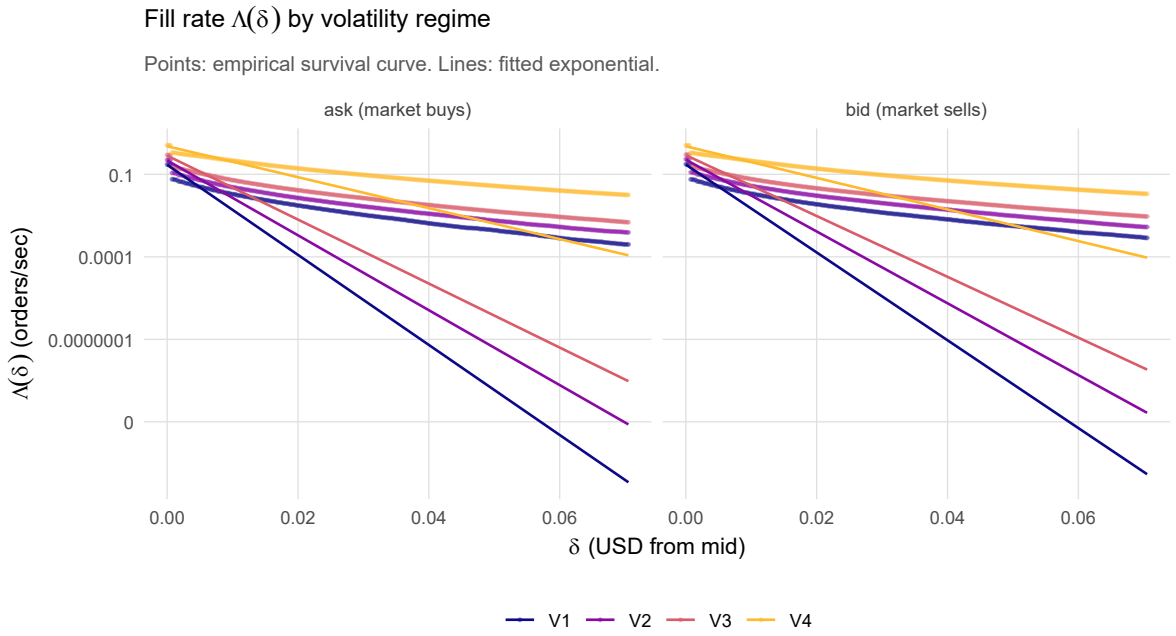

Figure 7: Fitted fill-rate curves per volatility regime. The market maker's trade-off is the whole curve, not either parameter alone: quoting at a fixed depth earns a different fill rate in each regime.

4.2.2 What the regimes imply for quoting

Figure 7 is the honest object but a hard one to act on. Collapsing each curve to the two numbers a quoting engine would actually use — where to rest, and what that earns — makes the regime effect legible.

```
vol_opt <- vol_fit[, {
  ds <- 1 / kappa
  # Envelope theorem: at the maximiser the derivative in delta vanishes, so
  # delta* can be held fixed when propagating the coefficient uncertainty
  # through to the edge. The band is therefore just the band on Lambda(delta*).
  b <- intensity_band(b0, b1, v00, v01, v11, ds)
```

```
  .(delta_star = ds / TICK_SIZE,
    ds_lo      = (1 / kappa_hi) / TICK_SIZE,   # reciprocal flips the interval
    ds_hi      = (1 / kappa_lo) / TICK_SIZE,
    edge       = ds * b$lambda * 3600,         # USD per hour, per unit quoted
    edge_lo    = ds * b$lo * 3600,
    edge_hi    = ds * b$hi * 3600)
}, by = .(vol_regime, side)]

opt_panels <- rbind(
  vol_opt[, .(vol_regime, side, value = delta_star, lo = ds_lo, hi = ds_hi,
              panel = "Optimal depth (ticks)")],
  vol_opt[, .(vol_regime, side, value = edge, lo = edge_lo, hi = edge_hi,
              panel = "Edge at that depth (USD/hour)")]
)

ggplot(opt_panels, aes(x = vol_regime, y = value, colour = side, group = side)) +
  geom_line(alpha = 0.6) +
  geom_pointrange(aes(ymin = lo, ymax = hi), size = 0.35) +
  facet_wrap(~ panel, scales = "free_y") +
  scale_colour_manual(values = c(col_data, col_model)) +
  labs(
    title    = "What each volatility regime prescribes",
    subtitle = sprintf("%s, %s - delta* = 1/kappa, edge = delta* x Lambda(delta*)",
                       SYMBOL, MONTH),
    x = "Volatility regime (low to high)", y = NULL
  )
```

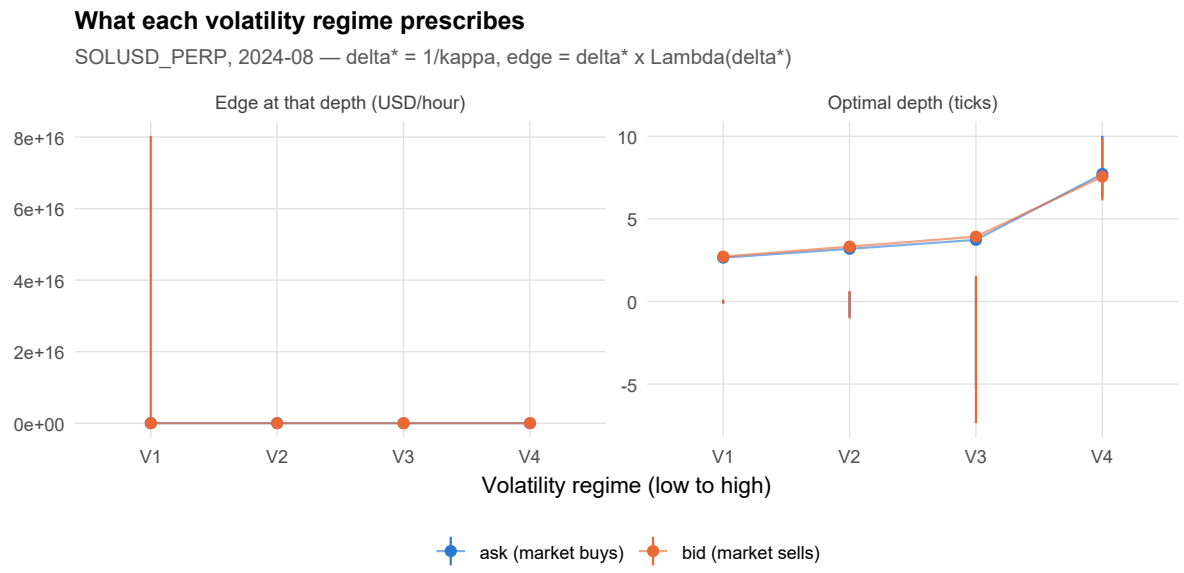

Figure 8: Risk-neutral optimal depth and the expected edge it earns, per volatility regime and side, with 95% intervals. Depth is in ticks; edge is gross of adverse selection and inventory cost. If the depth panel is flat within its intervals then the regimes prescribe the same quote and the split has no effect on the decision, however different the parameters look individually.

On the ask side that runs from 2.65 ticks in V1 to 7.72 ticks in V4, earning 0.78 and 10.72 USD per hour per unit quoted respectively.

This is the panel that decides whether the regime split is worth its complexity, and it is a stricter test than Figure 6. A and κ can both move convincingly while $\delta^* = 1/\kappa$ barely does, in which case the maker quotes at the same depth regardless and the conditioning only changes her forecast of how *often* she trades — useful for sizing and for expected P&L, but not for where the order goes.

4.3 Assessing the regime split

Three things to read off the output above.

Does κ actually move with volatility? The test is whether the regime-to-regime spread in $\hat{\kappa}$ is large compared to the (overdispersion-corrected) intervals in Figure 6. The expected sign is κ falling as volatility rises: in fast markets takers cross further, so the distance-from-mid distribution has a fatter tail and depth buys less protection. If the intervals overlap across all regimes, then volatility is not adding information over the pooled fit and the extra complexity is not earning its place.

Does A move in the same direction? A should rise with volatility — more arrivals at every depth. Note that A and κ moving together is not double-counting: they answer different questions. A sets how much flow there is; κ sets how quickly it thins with depth. The quantity that matters for quoting is the whole curve in Figure 7, and the two parameters can push it in opposing directions at different depths.

Is the exponential still a good fit inside each regime? Pooling across regimes is itself a mixture of exponentials, which is fatter-tailed than any single exponential — so some of the curvature visible in the pooled Figure 1 should *disappear* once the regimes are separated. If it does, that is direct evidence the regime split is capturing something real rather than slicing noise. If the within-regime curves are just as bent, the departure from exponential is intrinsic and the parameterisation itself is the limitation, not the conditioning.

```
# Pearson dispersion: lower means the exponential describes the within-group
# counts better. Pooled vs. regime-conditioned, over the same support.
fit_quality <- rbind(
  glm_fit[, .(side, cut = "pooled", dispersion)],
  vol_fit[, .(cut = "by volatility regime",
              dispersion = mean(dispersion)), by = side]
)

dcast(fit_quality, side ~ cut, value.var = "dispersion")
```

```
Key: <side>
                 side by volatility regime   pooled
               <fctr>                <num>    <num>
1:  ask (market buys)              5040807 41687.82
2: bid (market sells)              7805785 48731.69
```

A meaningful drop in dispersion from `pooled` to `by volatility regime` says the volatility conditioning is absorbing real heterogeneity. A flat or rising number says it is not, and the pooled estimate should be preferred on parsimony grounds.

The diagnostic does behave as advertised, which is worth knowing before reading anything into it. On a synthetic sample built from four exponentials with κ falling from 2000 to 1100 and A rising from 1.0 to 3.0, pooling gives $\phi \approx 17$–21, while fitting the same draws regime by regime returns $\phi \approx 1$ and recovers every (A, κ) to within about a percent. A drop of that order here would be strong evidence; a drop from 40 to 38 would not.

5 Still to do

- Validate out of sample on `2024-09`: hold the August (A, κ) fixed, predict September's fill-rate curve, and check both the pooled and the regime-conditioned fit against it. Month-to-month parameter drift is likely to dwarf the sampling intervals reported here, and until that is measured the intervals describe only how well the month was fitted, not how well the next one will be predicted.
- Aggregate on orderflow imbalance rather than on the raw arrival stream, which tends to reduce noise and improve out-of-sample fit.
- Add an intraday time-of-day cut: the other natural conditioning variable, and one partly confounded with the volatility cut used here.
- Quantify adverse selection: Section 3.5 prices the edge gross of where the mid goes next, which is the optimistic half of the trade.

References

Avellaneda, M., and S. Stoikov. 2008. "High-Frequency Trading in a Limit Order Book." *Quantitative Finance* 8 (3): 217–24.

Guéant, O., C.-A. Lehalle, and J. Fernandez-Tapia. 2013. "Dealing with the Inventory Risk: A Solution to the Market Making Problem." *Mathematics and Financial Economics* 7 (4): 477–507.